16 Multi-Stage Stochastic Programming

Stochastic programming is a computational approach to stochastic optimization. Stochastic programs have been studied for several decades (see, e.g., Birge and Louveaux, 1997; Shapiro et al., 2009). Typically, the approach hinges on a reformulation of the stochastic optimization problem as a deterministic one via a *scenario tree*. Computational and algorithmic advances have made stochastic programming techniques applicable to various classes of real-world problems.

16.1 Multi-Stage Stochastic Programming

Multi-stage stochastic optimization can be seen as a generalization of the generic class of stochastic optimization model discussed in Chapter 10. Let $0, 1, \ldots, T$ index a set of stages where decisions are to be made. Assume that between two consecutive stages $t-1$ and t some random outcome ω_t is revealed. At each stage $t = 0, 1, \ldots, T$ we make a set of *non-anticipatory* decisions $\mathbf{x}_t$ that can only depend on the random information revealed up until that stage. Schematically, the process can be seen as follows:

$$\underset{\mathbf{x}_0}{\text{decision}} \rightsquigarrow \underset{\text{draw } \omega_1}{\text{random}} \rightsquigarrow \underset{\mathbf{x}_1}{\text{decision}} \rightsquigarrow \underset{\text{draw } \omega_2}{\text{random}} \rightsquigarrow \cdots \rightsquigarrow \underset{\text{draw } \omega_T}{\text{random}} \rightsquigarrow \underset{\mathbf{x}_T}{\text{decision}}$$

A multi-stage stochastic minimization problem is the following kind of *multi-fold* version of the two-stage stochastic model (10.2) discussed in Chapter 10:

$$\begin{aligned} \min_{\mathbf{x}_0} \quad & g_0(\mathbf{x}_0) + \mathbb{E}[Q_1(\mathbf{x}_0, \omega_1)] \\ & \mathbf{x}_0 \in \mathcal{X}_0, \end{aligned} \tag{16.1}$$

where the recourse term $Q_1(\mathbf{x}_0, \omega_1)$ similarly depends on the decisions to be made at later stages:

$$\begin{aligned} Q_1(\mathbf{x}_0, \omega_1) := \min_{\mathbf{x}_1} \quad & g_1(\mathbf{x}_1, \omega_1) + \mathbb{E}[Q_2(\mathbf{x}_1, \omega_2)] \\ & \mathbf{x}_1 \in \mathcal{X}_1(\mathbf{x}_0, \omega_1), \end{aligned}$$

with

$$\begin{aligned} Q_t(\mathbf{x}_{t-1}, \omega_t) := \min_{\mathbf{x}_t} \quad & g_t(\mathbf{x}_t, \omega_t) + \mathbb{E}[Q_{t+1}(\mathbf{x}_t, \omega_{t+1})] \\ & \mathbf{x}_t \in \mathcal{X}_t(\mathbf{x}_{t-1}, \omega_t), \end{aligned}$$

for $t = 2, \ldots, T-1$, and the last-stage recourse term $Q_T(\mathbf{x}_{T-1}, \omega_T)$ is of the form

$$Q_T(\mathbf{x}_{T-1}, \omega_T) := \min_{\mathbf{x}_T} \quad g_T(\mathbf{x}_T, \omega_T) \\ \mathbf{x}_T \in \mathcal{X}_T(\mathbf{x}_{T-1}, \omega_T).$$

The multi-stage optimization problem (16.1) can also be written as

$$\min_{\mathbf{x}_0 \in \mathcal{X}_0} g_0(\mathbf{x}_0) + \mathbb{E}\left[\min_{\mathbf{x}_1 \in \mathcal{X}_1(\mathbf{x}_0, \omega_1)} g_1(\mathbf{x}_1, \omega_1) + \cdots + \mathbb{E}\left[\min_{\mathbf{x}_T \in \mathcal{X}_T(\mathbf{x}_{T-1}, \omega_T)} g_T(\mathbf{x}_T, \omega_T)\right]\right].$$

Consider the special case of linear multi-stage stochastic optimization, where the components are linear. More precisely, each $g_t(\mathbf{x}_t, \omega_t) = \mathbf{c}_t^\mathsf{T}\mathbf{x}_t$ for some vector $\mathbf{c}_t$ and each inter-temporal constraint $\mathbf{x}_t \in \mathcal{X}_t(\mathbf{x}_{t-1}, \omega_t)$ is of the form

$$\mathbf{B}_t\mathbf{x}_{t-1} + \mathbf{A}_t\mathbf{x}_t = \mathbf{b}_t, \;\; \mathbf{x}_t \geq 0,$$

where $\omega_t = (\mathbf{c}_t, \mathbf{A}_t, \mathbf{B}_t, \mathbf{b}_t)$ is only revealed at stage t. In this case we often write

$$\begin{array}{llccccccccc} \min\limits_{\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T} & \mathbb{E}\big[\mathbf{c}_0^\mathsf{T}\mathbf{x}_0 & + & \mathbf{c}_1^\mathsf{T}\mathbf{x}_1 & + & \cdots & + & \mathbf{c}_T^\mathsf{T}\mathbf{x}_T\big] & & \\ \text{s.t.} & \mathbf{A}_0\mathbf{x}_0 & & & & & & & = & \mathbf{b}_0 \\ & \mathbf{B}_1\mathbf{x}_0 & + & \mathbf{A}_1\mathbf{x}_1 & & & & & = & \mathbf{b}_1 \\ & & & \mathbf{B}_2\mathbf{x}_1 & + & \mathbf{A}_2\mathbf{x}_2 & & & = & \mathbf{b}_2 \\ & & & & & \vdots & & \vdots & & \\ & & & & & \mathbf{B}_T\mathbf{x}_{T-1} & + & \mathbf{A}_T\mathbf{x}_T & = & \mathbf{b}_T \\ & \mathbf{x}_0, & & \mathbf{x}_1, & & \ldots & & \mathbf{x}_T & \geq & 0. \end{array} \tag{16.2}$$

Example 16.1 (Financial planning example) Assume an investor has initial wealth W_0 at $t = 0$. At stage t she can invest in two asset classes: bonds and stocks. The (random) gross return on bonds from time $t-1$ to t is $R_{b,t}$ and the (random) gross return on stocks from $t-1$ to t is $R_{s,t}$. Assume that the investor needs to meet liabilities L_t at times $t = 1, \ldots, T$. She wants to maximize her expected wealth at time T (after covering the liabilities). Assume no shorting is allowed.

Formulation of the financial planning example

Variables:

x_t: amount of money invested in bonds at stage t, for $t = 0, \ldots, T-1$;
y_t: amount of money invested in stocks at stage t, for $t = 0, \ldots, T-1$;
W_T: wealth at time T

$$\begin{array}{ll} \max & \mathbb{E}(W_T) \\ \text{s.t.} & x_0 + y_0 = W_0 \\ & R_{b,t}x_{t-1} + R_{s,t}y_{t-1} = L_t + x_t + y_t, \; t = 1, \ldots, T-1 \\ & R_{b,T}x_{T-1} + R_{s,T}y_{T-1} = L_T + W_T \\ & x_t, y_t \geq 0, \; t = 0, 1, \ldots, T-1 \\ & W_T \geq 0. \end{array}$$

Notice that in this model the parameters $R_{b,t}$ and $R_{s,t}$ are unknown prior to time t.

16.2 Scenario Optimization

As discussed in Chapter 10 for the two-stage case, a multi-stage optimization model can be recast as a *deterministic equivalent* if each of the random outcomes has a discrete distribution. In this case for each stage $t = 1, 2, \ldots, T$ there is a finite set of possible values or realizations $\{\omega_t^1, \ldots, \omega_t^S\}$ for the random outcome ω_t. These sets of realizations can be described by an *event tree* as depicted in Figure 16.1 for a problem with three stages. In this particular tree, the random variables ω_1 and ω_2 have two- and five-valued discrete distributions respectively. The tree structure is associated with the discrete filtration generated by the discrete-time random process $\omega_{[t]} := (\omega_1, \omega_2, \ldots, \omega_t),\ t = 1, \ldots, T$. In particular, each possible value of ω_2 has a unique *predecessor* value of ω_1. We further elaborate on this tree structure below.

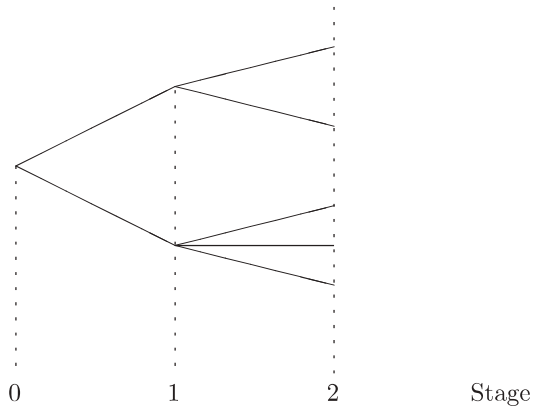

Figure 16.1 Event tree for a three-stage model

The set of scenarios described by the event tree in turn yield a deterministic equivalent of a multi-stage stochastic optimization model. We next illustrate this equivalence for the stochastic optimization model described in Example 16.1 in a particularly simple event tree. We subsequently describe the deterministic equivalent for linear multi-stage stochastic programs in more general event trees.

Example 16.2 (Financial planning revisited) Consider the model described in Example 16.1. Suppose $T = 2$ and there are two equally likely outcomes ("H" and "T") for the joint returns $(R_{b,t}, R_{s,t})$ over each period, say

$$(R_{b,t}(H), R_{s,t}(H)) = (1.14, 1.25) \ \text{ and } \ (R_{b,t}(T), R_{s,t}(T)) = (1.1, 1.06).$$

Figure 16.2 illustrates the corresponding scenario tree. In this event tree the labels "H" and "T" on the edges indicate the specific outcome between two

consecutive stages. Observe that each of the four scenarios HH, HT, TH, TT in the event tree occurs with probability 1/4.

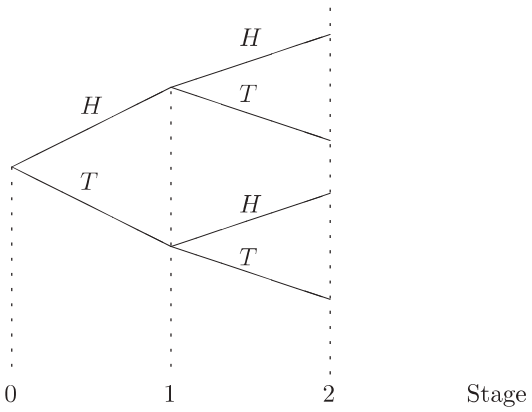

Figure 16.2 Event tree for financial planning model

The scenario tree in turn yields a deterministic equivalent formulation for the financial planning stochastic optimization model. In the deterministic equivalent the stage 0 decisions are made at the root of the tree and thus may not depend on any of the random outcomes. The stage 1 decisions may depend on the initial path H or T realized up to stage 1 in the event tree. Finally, the stage 2 decisions may depend on the path HH, HT, TH, or TT realized up to stage 2 in the event tree. The corresponding adaptiveness of the variables and constraints is explicitly reflected in the following deterministic equivalent formulation.

Scenario optimization model

Variables:

x_0, y_0: money in bonds and stocks at $t = 0$;
$x_1(H), x_1(T), y_1(H), y_1(T)$: money in bonds and stocks at $t = 1$;
$W_2(HH), W_2(HT), W_2(TH), W_2(TT)$: wealth at $t = 2$

$$\max \quad \frac{1}{4} \cdot (W_2(HH) + W_2(HT) + W_2(TH) + W_2(TT))$$

$$\text{s.t.} \quad x_0 + y_0 = W_0 \qquad \text{(stage 0)}$$

$$1.14x_0 + 1.25y_0 = L_1 + x_1(H) + y_1(H) \qquad \text{(stage 1, path } H\text{)}$$
$$1.1x_0 + 1.06y_0 = L_1 + x_1(T) + y_1(T) \qquad \text{(stage 1, path } T\text{)}$$

$$1.14x_1(H) + 1.25y_1(H) = L_2 + W_2(HH) \qquad \text{(stage 2, } HH\text{)}$$
$$1.1x_1(H) + 1.06y_1(H) = L_2 + W_2(HT) \qquad \text{(stage 2, } HT\text{)}$$
$$1.14x_1(T) + 1.25y_1(T) = L_2 + W_2(TH) \qquad \text{(stage 2, } TH\text{)}$$
$$1.1x_1(T) + 1.06y_1(T) = L_2 + W_2(TT) \qquad \text{(stage 2, } TT\text{)}$$

$$x_0, y_0, x_1(H), x_1(T), y_1(H), y_1(T) \geq 0$$
$$W_2(HH), W_2(HT), W_2(TH), W_2(TT) \geq 0.$$

The above scenario optimization approach is quite flexible. In particular, consider a variation of the above financial planning model where the objective is $\max \mathbb{E}\left(U(W_T)\right)$ for some concave utility function $U(W)$. The corresponding deterministic equivalent has exactly the same variables and constraints as the one above and the following objective:

$$\max \frac{1}{4}\left(U(W_2(HH)) + U(W_2(TH)) + U(W_2(HT)) + U(W_2(TT))\right).$$

Furthermore, if $U(\cdot)$ is piecewise linear, then the problem can be recast as a linear program. (See Exercise 16.1.)

Consider now the general multi-stage linear stochastic program (16.2). Suppose each random vector $\omega_t = (\mathbf{c}_t, \mathbf{A}_t, \mathbf{B}_t, \mathbf{b}_t)$ has a discrete distribution and consider their event tree representation. The description of the deterministic equivalent relies on the following notation. Let

$$\Omega_t := \{\omega_t^k = (\mathbf{c}_t^k, \mathbf{A}_t^k, \mathbf{B}_t^k, \mathbf{b}_t^k) : k = 1, \ldots, S_t\}$$

be the set of possible realizations of the random variable ω_t for some integer $S_t \geq 1$ and for each stage $t = 1, \ldots, T$. Let $p_t^k = \mathbb{P}(\omega_t = \omega_t^k)$, with $k = 1, \ldots, S_t$, $t = 1, \ldots, T$. The set $\Omega_t = \{\omega_t^1, \ldots, \omega_t^{S_t}\}$ corresponds to the nodes in layer t of the event tree, which can be conveniently denoted $(t, 1), \ldots, (t, S_t)$ as illustrated in Figure 16.3.

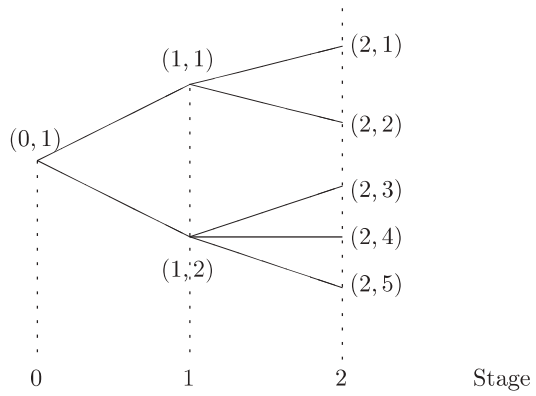

Figure 16.3 Event tree for a three-stage model with node labels

Observe that in the event tree there is always a single root node $(0, 1)$ in layer 0. Furthermore, for each $t = 1, \ldots, T-1$ each node (t, k) has a *unique* predecessor $(t-1, \hat{k})$ in the immediately preceding layer of the tree. For instance, in the event tree depicted in Figure 16.3 the predecessors of each of the nodes in layer 2 is a unique node in layer 1 as follows

$$\hat{1} = \hat{2} = 1, \ \hat{3} = \hat{4} = \hat{5} = 2.$$

Observe that the probability of a non-terminal node equals the combined probability of its direct descendants; that is, for $t = 1, \ldots, T$ and for every node

$(t-1,\ell)$ we have

$$p_{t-1}^{\ell} = \sum_{(t,k):\hat{k}=\ell} p_t^k.$$

Consider the multi-stage linear stochastic program (16.2) and assume the random outcomes are described via a suitable event tree. We next detail the variables, objective, and constraints of the corresponding deterministic linear program equivalent.

Variables: Stage 0 variables: $\mathbf{x}_0$.

Stage t variables can be adapted to the S_t possible paths up to stage t; that is,

$$\mathbf{x}_t^k, \;\; k = 1, \ldots, S_t.$$

Objective: The deterministic equivalent of the objective function

$$\min \mathbb{E}\big[\mathbf{c}_0^\mathsf{T}\mathbf{x}_0 + \mathbf{c}_1^\mathsf{T}\mathbf{x}_1 + \cdots + \mathbf{c}_T^\mathsf{T}\mathbf{x}_T\big] = \min \mathbb{E}\left[\mathbf{c}_0^\mathsf{T}\mathbf{x}_0 + \sum_{t=1}^{T} \mathbf{c}_t^\mathsf{T}\mathbf{x}_t\right]$$

is

$$\min \mathbf{c}_0^\mathsf{T}\mathbf{x}_0 + \sum_{t=1}^{T}\sum_{k=1}^{S_t} p_t^k (\mathbf{c}_t^k)^\mathsf{T}\mathbf{x}_t^k.$$

Constraints: The deterministic equivalent of each inter-temporal constraint

$$\mathbf{B}_t\mathbf{x}_{t-1} + \mathbf{A}_t\mathbf{x}_t = \mathbf{b}_t, \;\; \mathbf{x}_t \geq 0,$$

is the set of constraints

$$\mathbf{B}_t^k\mathbf{x}_{t-1}^{\hat{k}} + \mathbf{A}_t^k\mathbf{x}_t^k = \mathbf{b}_t, \;\; \mathbf{x}_t^k \geq 0, \;\; \text{for} \;\; k = 1, \ldots, S_t.$$

Observe that these constraints link the stage t variables $\mathbf{x}_t^k$ associated with the layer t nodes (t,k) with the variable associated with their predecessor $(t-1,\hat{k})$.

Thus the complete deterministic equivalent of (16.2) is as follows:

$$\begin{array}{lllllll}
\min & c_0^\mathsf{T}\mathbf{x}_0 + \displaystyle\sum_{k=1}^{S_1} p_1^k(\mathbf{c}_1^k)^\mathsf{T}\mathbf{x}_1^k + & \cdots & + \displaystyle\sum_{k=1}^{S_T} p_T^k(\mathbf{c}_T^k)^\mathsf{T}\mathbf{x}_T^k & & (16.3)\\
\text{s.t.} & \mathbf{A}_0\mathbf{x}_0 & & & = \mathbf{b}_0 & & \\
& \mathbf{B}_1^k\mathbf{x}_0 + \quad \mathbf{A}_1^k\mathbf{x}_1^k & & & = \mathbf{b}_1^k, \;\; k = 1, \ldots, S_1 & & \\
& \quad \mathbf{B}_2^k\mathbf{x}_1^{\hat{k}} + \quad \mathbf{A}_2^k\mathbf{x}_2^k & & & = \mathbf{b}_2, \;\; k = 1, \ldots, S_2 & & \\
& \qquad\qquad \vdots & & & \vdots & & \\
& \qquad\qquad \mathbf{B}_T^k\mathbf{x}_{T-1}^{\hat{k}} + & & \mathbf{A}_T^k\mathbf{x}_T^k & = \mathbf{b}_T^k, \;\; k = 1, \ldots, S_T & & \\
& \mathbf{x}_0, \qquad\qquad \mathbf{x}_1^k, & \cdots & & \mathbf{x}_T^k \geq 0. & &
\end{array}$$

For example, if $T = 2$ and the event tree is as depicted in Figure 16.3, then the deterministic equivalent is

$$
\begin{array}{lllllllllllllll}
\min & \mathbf{c}_0^\mathsf{T}\mathbf{x}_0 & + & p_1^1(\mathbf{c}_1^1)^\mathsf{T}\mathbf{x}_1^1 & + & p_1^2(\mathbf{c}_1^2)^\mathsf{T}\mathbf{x}_1^2 & + & p_2^1(\mathbf{c}_2^1)^\mathsf{T}\mathbf{x}_2^1 & + & p_2^2(\mathbf{c}_2^2)^\mathsf{T}\mathbf{x}_2^2 & + & p_2^3(\mathbf{c}_2^3)^\mathsf{T}\mathbf{x}_2^3 \\
 & & & & & & & & & + \; p_2^4(\mathbf{c}_2^4)^\mathsf{T}\mathbf{x}_2^4 & + & p_2^5(\mathbf{c}_2^5)^\mathsf{T}\mathbf{x}_2^5 \\
\text{s.t.} & \mathbf{A}_0\mathbf{x}_0 & & & & & & & & & & & = \mathbf{b}_0 \\
 & \mathbf{B}_1^1\mathbf{x}_0 + \mathbf{A}_1^1\mathbf{x}_1^1 & & & & & & & & & & & = \mathbf{b}_1^1 \\
 & \mathbf{B}_1^2\mathbf{x}_0 + & & \mathbf{A}_1^2\mathbf{x}_1^2 & & & & & & & & & = \mathbf{b}_1^2 \\
 & & \mathbf{B}_2^1\mathbf{x}_1^1 & & + & \mathbf{A}_2^1\mathbf{x}_2^1 & & & & & & & = \mathbf{b}_2^1 \\
 & & \mathbf{B}_2^2\mathbf{x}_1^1 & & + & & & \mathbf{A}_2^2\mathbf{x}_2^2 & & & & & = \mathbf{b}_2^2 \\
 & & & \mathbf{B}_2^3\mathbf{x}_1^2 & + & & & & & \mathbf{A}_2^3\mathbf{x}_2^3 & & & = \mathbf{b}_2^3 \\
 & & & \mathbf{B}_2^4\mathbf{x}_1^2 & + & & & & & & & \mathbf{A}_2^4\mathbf{x}_2^4 & = \mathbf{b}_2^4 \\
 & & & \mathbf{B}_2^5\mathbf{x}_1^2 & + & & & & & & & & \mathbf{A}_2^5\mathbf{x}_2^5 = \mathbf{b}_2^5 \\
 & \mathbf{x}_0, & \mathbf{x}_1^1, & \mathbf{x}_1^2, & & \mathbf{x}_2^1, & & \mathbf{x}_2^2, & & \mathbf{x}_2^3, & & \mathbf{x}_2^4, & \mathbf{x}_2^5 \geq \mathbf{0}.
\end{array}
$$

Observe that the constraint matrix in the above model has the following structure:

$$
\begin{bmatrix}
\mathbf{A}_0 & & & & & & & \\
\mathbf{B}_1^1 & \mathbf{A}_1^1 & & & & & & \\
\mathbf{B}_1^2 & & \mathbf{A}_1^2 & & & & & \\
 & \mathbf{B}_2^1 & & \mathbf{A}_2^1 & & & & \\
 & \mathbf{B}_2^2 & & & \mathbf{A}_2^2 & & & \\
 & & \mathbf{B}_2^3 & & & \mathbf{A}_2^3 & & \\
 & & \mathbf{B}_2^4 & & & & \mathbf{A}_2^4 & \\
 & & \mathbf{B}_2^5 & & & & & \mathbf{A}_2^5
\end{bmatrix}.
$$

The constraint matrix for the general deterministic equivalent (16.3) has a similar type of structure.

There are alternative ways of labeling the nodes and branches in the event tree. In particular, the simple binary branching in Example 16.2 readily suggests the natural edge labeling depicted in Figure 16.2. In that case the nodes in layer t can alternatively be labeled via the t-long sequence of labels along the path from the root node. This kind of edge labeling may appear more intuitive but it could become cumbersome in other cases when there are different numbers of branches at each non-terminal node.

Note that the size of the deterministic equivalent of a multi-stage stochastic program increases rapidly with the number of stages. For example, for a problem with 11 stages and a binary event tree, there are $2^{10} = 1024$ scenarios and therefore the linear program (16.3) may have several thousand constraints and variables, depending on the number of variables and constraints at each node. Modern commercial codes can handle such large linear programs, but a moderate increase in the number of stages or in the number of branches at each stage could

make (16.3) too large to solve by standard linear programming solvers. When this happens, it is critical to exploit the special structure of (16.3) to solve the model efficiently.

The Benders decomposition (or L-shaped method) introduced in Section 10.5 can also be used for multi-stage problems (16.3) in a straightforward way: The stages are partitioned into a first set that gives rise to the "master problem" and a second set that gives rise to the "recourse problems". For example in a six-stage problem, the variables of the first two stages could define the master problem. When these variables are fixed, (16.3) decomposes into separate linear programs each involving variables of the last four stages. The solutions of these recourse linear programs provide optimality or feasibility cuts that can be added to the master problem. As discussed in Section 10.5, upper and lower bounds are computed at each iteration and the algorithm stops when the difference drops below a given tolerance. Using this approach, Gondzio and Kouwenberg (2001) were able to solve an asset liability management problem with over four million scenarios, whose linear programming formulation (16.3) had 12 million constraints and 24 million variables. This linear program was so large that storage space on the computer became an issue. The scenario tree had six levels and 13 branches at each node. In order to apply Benders' decomposition, Gondzio and Kouwenberg divided the six-period problem into a first-stage problem containing the first three periods and a second-stage problem containing periods four to six. This resulted in 2197 recourse linear programs, each involving 2197 scenarios. These recourse linear programs were solved by an interior-point algorithm. Note that Benders' decomposition is ideally suited for parallel computations since the recourse linear programs can be solved simultaneously. When the solution of all the recourse linear programs is completed (which takes the bulk of the time), the master problem is then solved on one processor while the other processors remain idle temporarily. Gondzio and Kouwenberg tested a parallel implementation on a computer with 16 processors and they obtained an almost perfect speedup, that is a speedup factor of almost k when using k processors.

16.3 Scenario Generation

A key aspect of multi-stage stochastic programming is the generation of scenarios so that the deterministic equivalent formulation (16.3) accurately represents the underlying stochastic optimization problem.

There are two separate issues. First, one needs to model the correlation over time among the random parameters. For a pension fund, such a model might relate wage inflation (a random parameter that influences the liability side) to interest rates and stock prices (random parameters that influence the asset side). Below we discuss a simple autoregressive model that can be used for this purpose. A second issue is the construction of a scenario tree from these inter-temporal statistical models: A finite number of scenarios must reflect as accurately as

possible the random processes modeled in the previous step, suggesting the need for a large number of scenarios. On the other hand, the linear program (16.3) can only be solved if the size of the scenario tree is reasonable, suggesting a limited number of scenarios. To reconcile these two conflicting objectives, it might be crucial to use variance reduction techniques. We address these issues in this section.

Autoregressive Model

In order to generate the random parameters underlying the stochastic program, one needs to construct an economic model reflecting the correlation between the parameters. Historical data may be available. The goal is to generate meaningful time series for constructing the scenarios. One approach is to use an autoregressive model.

Specifically, if $\mathbf{r}_t$ denotes the random vector of parameters in period t, an *autoregressive model* is defined by

$$\mathbf{r}_t = \mathbf{D}_0 + \mathbf{D}_1\mathbf{r}_{t-1} + \cdots + \mathbf{D}_p\mathbf{r}_{t-p} + \boldsymbol{\epsilon}_t,$$

where p is the number of lags used in the regression, $\mathbf{D}_0, \mathbf{D}_1, \ldots, \mathbf{D}_p$ are time-independent constant matrices, which are estimated through statistical methods such as maximum likelihood, and $\boldsymbol{\epsilon}_t$ is a vector of i.i.d. random disturbances with mean zero.

To illustrate this, consider a problem where the vector $\mathbf{r}_t$ consists of three random parameters: s_t, b_t, and m_t are the rates of return of stocks, bonds, and the money market, respectively, in year t. An autoregressive model with $p = 1$ has the form:

$$\begin{bmatrix} s_t \\ b_t \\ m_t \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} + \begin{bmatrix} d_{11} & d_{12} & d_{13} \\ d_{21} & d_{22} & d_{23} \\ d_{31} & d_{32} & d_{33} \end{bmatrix} \begin{bmatrix} s_{t-1} \\ b_{t-1} \\ m_{t-1} \end{bmatrix} + \begin{bmatrix} \epsilon_t^s \\ \epsilon_t^b \\ \epsilon_t^m \end{bmatrix}, \qquad t = 2, \ldots, T.$$

Assuming independent error terms ϵ_t^s, ϵ_t^b, and ϵ_t^m, and using historical data, one can find the parameters d_1, d_{11}, d_{12}, d_{13} in the first equation,

$$s_t = d_1 + d_{11}s_{t-1} + d_{12}b_{t-1} + d_{13}m_{t-1} + \epsilon_t^s,$$

using standard linear regression tools that minimize the sum of squared errors ϵ_t^s. Useful statistics, such as the standard error σ_s of the estimates s_t, can also be obtained. Similarly for b_t and m_t.

Constructing Scenario Trees

The random distributions relating the various parameters of a stochastic program must be discretized to generate a set of scenarios that is adequate for its deterministic equivalent. Too few scenarios may lead to approximation errors. On the other hand, too many scenarios will lead to an explosion in the size of

the scenario tree, leading to an excessive computational burden. In this section, we discuss a simple random sampling approach and two variance reduction techniques: adjusted random sampling and tree fitting. Unfortunately, scenario trees constructed by these methods could contain spurious arbitrage opportunities. We end this section with a procedure to test that this does not occur.

Random Sampling

One can generate scenarios directly from the autoregressive model introduced in the previous section:

$$\mathbf{r}_t = \mathbf{D}_0 + \mathbf{D}_1\mathbf{r}_{t-1} + \cdots + \mathbf{D}_p\mathbf{r}_{t-p} + \boldsymbol{\epsilon}_t,$$

where $\epsilon_t \sim N(\mathbf{0}, \Sigma)$ are independently distributed multivariate normal distributions with mean 0 and covariance matrix Σ.

In our example with three random parameters s_t, b_t, and m_t, and independent error terms ϵ_t^s, ϵ_t^b, ϵ_t^m, the matrix Σ is a 3×3 diagonal matrix, with diagonal entries σ_s, σ_b, σ_m. Thirty branches or so may be needed to get a reasonable approximation of the distribution of the rates of return in stage 1. For a problem with three stages, 30 branches at each stage represent 27,000 scenarios. With more stages, the size of the linear program (16.3) explodes. Kouwenberg (2001) performed tests on scenario trees with fewer branches at each node (such as a five-stage problem with branching structure 10–6–6–4–4, meaning ten branches at the root, then six branches at each node in the next stage and so on) and he concluded that random sampling on such trees leads to unstable investment strategies. This occurs because the approximation error made by representing parameter distributions by random samples can be significant in a small scenario tree. As a result the optimal solution of (16.3) is not optimal for the actual parameter distributions. How can one construct a scenario tree that more accurately represents these distributions, without blowing up the size of the linear program (16.3)?

Adjusted Random Sampling

An easy way of improving upon random sampling is as follows. Assume that each node of the scenario tree has an even number $K = 2k$ of branches. Instead of generating $2k$ random samples from the autoregressive model, generate k random samples only and use the negative of their error terms to compute the values on the remaining k branches. This will fit all the odd moments of the distributions correctly. In order to fit the variance of the distributions as well, one can scale the sampled values. The sampled values are all scaled by a multiplicative factor until their variance fits that of the corresponding parameter.

Tree Fitting

How can one best approximate a continuous distribution by a discrete distribution with K values? In other words, how should one choose values v_k and their probabilities p_k, for $k = 1, \ldots, K$, in order to approximate the given distribution

as accurately as possible? A natural answer is to match as many of the moments as possible. In the context of a scenario tree, the problem is somewhat more complicated since there are several correlated parameters at each node and there is interdependence between periods as well. Hoyland and Wallace (2001) propose to formulate this fitting problem as a nonlinear program. The fitting problem can be solved either at each node separately or on the overall tree. We explain the fitting problem at a node. Let S_l be the values of the statistical properties of the distributions that one desires to fit, for $l = 1, \ldots, s$. These might be the expected values of the distributions, the correlation matrix, the skewness, and kurtosis. Let $\mathbf{v}_k$ and p_k denote the vector of values on branch k and its probability, respectively, for $k = 1, \ldots, K$. Let $f_l(\mathbf{v}, \mathbf{p})$ be the mathematical expression of property l for the discrete distribution (for example, the mean of the vectors $\mathbf{v}_k$, their correlation, skewness, and kurtosis). Each property has a positive weight w_l indicating its importance in the desired fit. Hoyland and Wallace formulate the fitting problem as

$$\begin{aligned} \min_{\mathbf{v},\mathbf{p}} \quad & \sum_l w_l (f_l(\mathbf{v}, \mathbf{p}) - S_l)^2 \\ \text{s.t.} \quad & \sum_k p_k = 1 \\ & \mathbf{p} \geq 0. \end{aligned} \tag{16.4}$$

One might want some statistical properties to match exactly. As an example, consider again the autoregressive model:

$$\mathbf{r}_t = \mathbf{D}_0 + \mathbf{D}_1 \mathbf{r}_{t-1} + \cdots + \mathbf{D}_p \mathbf{r}_{t-p} + \boldsymbol{\epsilon}_t,$$

where $\boldsymbol{\epsilon}_t \sim N(\mathbf{0}, \Sigma)$ are independently distributed multivariate normal distributions with mean 0 and covariance matrix Σ. To simplify notation, let us write $\boldsymbol{\epsilon}$ instead of $\boldsymbol{\epsilon}_t$. The random vector $\boldsymbol{\epsilon}$ has distribution $N(\mathbf{0}, \Sigma)$ and we would like to approximate this continuous distribution by a finite number of disturbance vectors $\boldsymbol{\epsilon}^k$ occuring with probability p_k, for $k = 1, \ldots, K$. Let ϵ_q^k denote the qth component of vector $\boldsymbol{\epsilon}^k$. One might want to fit the mean of $\boldsymbol{\epsilon}$ exactly and its covariance matrix as well as possible. In this case, the fitting problem is

$$\begin{aligned} \min_{\boldsymbol{\epsilon}^1, \ldots, \boldsymbol{\epsilon}^K, \mathbf{p}} \quad & \sum_{q=1}^{l} \sum_{r=1}^{l} \left(\sum_{k=1}^{K} p_k \epsilon_q^k \epsilon_r^k - \Sigma_{qr} \right)^2 \\ \text{s.t.} \quad & \sum_{k=1}^{K} p_k \boldsymbol{\epsilon}^k = \mathbf{0} \\ & \sum_k p_k = 1 \\ & \mathbf{p} \geq 0. \end{aligned}$$

Arbitrage-Free Scenario Trees

Approximating the continuous distributions of the uncertain parameters by a finite number of scenarios in the linear program (16.3) typically creates modeling

errors. In fact, if the scenarios are not chosen properly or if their number is too small, the supposed "linear programming equivalent" could be far from being equivalent to the original stochastic optimization problem. One of the most disturbing aspects of this phenomenon is the possibility of creating arbitrage opportunities when constructing the scenario tree. When this occurs, model (16.3) is flawed as it would be distorted by the arbitrage opportunities. Klaassen (2002) was the first to address this issue. In particular, he shows how arbitrage opportunities can be detected *ex post* in a scenario tree. See Exercise 16.3 for details. When arbitrage opportunities exist, a simple solution is to discard the scenario tree and to construct a new one with more branches. Klaassen also discusses what constraints to add to the nonlinear program (16.4) in order to preclude arbitrage opportunities *ex ante*. The additional constraints are nonlinear, thus increasing the difficulty of solving (16.4).

16.4 Exercises

Exercise 16.1 Consider the following variation of the "financial planning" problem discussed in Section 16.1:

- Assume there are four stages, 0 through 3 (i.e., $T = 3$).
- Over each period we have two equally likely outcomes for joint returns of bonds and stocks: (14%, 25%) and (10%, 6%).
- Initial wealth $W_0 = 55$. $L_1 = L_2 = 0$. Final liability $L_3 = 70$.

(a) Use a multi-stage scenario optimization approach to determine the sequence of investment decisions so that the liability is met at $T = 3$, and the expected value of the remaining wealth is maximized. Your investment decisions at each stage must be non-anticipatory. That is, they can only depend on the scenario path up to that stage.

(b) Modify your model to solve the following variation: Instead of the single liability at stage 3, the following sequence of liabilities must be met at stages 1, 2, 3:

$$L_1 = 20,\ L_2 = 20,\ L_3 = 25.$$

(c) Assume this time that $L_1 = L_2 = 0$ and that the final liability is $L_3 = 90$. Since this is too high, it is clear that, regardless of the investment decisions, the final wealth W_3 will be negative in some scenarios. Modify your model to maximize instead $\mathbb{E}(U(W_3))$, where the utility function $U(W)$ is as follows:

$$U(W) = \begin{cases} W & \text{if } W \geq 0 \\ 3W & \text{if } W < 0. \end{cases}$$

It is preferable for your model to be a linear program for computational purposes. For that purpose, you need to recast the objective

$$\max\ \mathbb{E}(U(W_T))$$

so that the resulting model is a linear program. To do so, observe that $U(W) = \min\{W, 3W\}$ and use a suitable set of new variables.

Exercise 16.2 Consider the following dynamic portfolio problem.

- At time $t = 0$ you have an initial endowment W_0.
- At time $t = 0, 1, \ldots, T-1$ you invest a fraction x_t of your wealth in a risky asset and the remaining fraction $1 - x_t$ in a risk-free asset. The risk-free and risky asset returns between t and $t+1$ are $r_{f,t+1}$ and $r_{s,t+1}$ respectively.
- At time $t = 1, \ldots, T$ you receive an exogenous and deterministic income of I_t.

Let W_t denote your wealth at time $t = 0, 1, \ldots, T$. Your goal is to maximize utility of final wealth $\mathbb{E}(U(W_T))$.

(a) Write the law of motion for W_t.

(b) Assume that between t and $t+1$ there are two equally likely outcomes H and T. The risky return $r_{s,t+1}$ in each of these scenarios is as follows:
- In outcome H: $r_{s,t+1} = 0.5$.
- In outcome T: $r_{s,t+1} = -0.4$.

Assume a zero risk-free return, $r_{f,t+1} = 0$, in both scenarios.

Suppose $T = 2$. Write down the "law of motion" for W_1 and W_2 in each relevant scenario using the above numerical values for $r_{f,t+1}, r_{s,t+1}$.

(c) Assume $W_0 = 1$ and $I_1 = I_2 = 0.1$. Use scenario optimization and Excel `Solver` or MATLAB to solve the two-period portfolio optimization problem

$$\max_{x_0, x_1} \ \mathbb{E}\left[\frac{W_2^{1-\gamma}}{1-\gamma}\right]$$

for $\gamma = 0.4, 0.7, 0.9$.

(d) Repeat part (c) but this time assume $W_0 = 1$ and $I_1 = I_2 = 0$.

(e) Repeat part (c) but this time assume $W_0 = 1$ and $I_1 = I_2 = 0.2$.

(f) Can you infer anything from the numerical results in (c), (d), and (e) about long-term investment when you know you will receive income along the investment horizon?

Exercise 16.3 Recall from Section 4.2 in Chapter 4 that an arbitrage opportunity is an opportunity to make money without any cost and without any risk. Consider a particular node at some stage $t - 1 \geq 0$ in a scenario tree whose set of immediate descendants in stage t is K. For each $k \in K$ let $\mathbf{r}^k \in \mathbb{R}^n$ denote the vector of asset returns of a set of n assets realized in branch k between stages $t-1$ and t.

(a) Show that an arbitrage opportunity exists if there is an asset allocation $\mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}$ such that

$$\sum_{j=1}^{n} x_j \leq 0, \ \sum_{j=1}^{n} (\mathbf{r}^k)^\mathsf{T} \mathbf{x} \geq \mathbf{0},$$

where at least one inequality is strict.

(b) Show that the condition in part (a) holds if and only if the following condition does not hold: there exist $y_k > 0$, for $k \in K$, such that

$$\sum_{k \in K} y_k \mathbf{r}^k = \mathbf{1}.$$

Hint: Apply the same kind of reasoning used in Section 4.2.

(c) Use part (a) and part (b) above to modify the nonlinear program (16.4) in order to formulate a fitting problem at a node that does not contain any arbitrage opportunities.